Guide technique du projet — détection et autocapture de la CIN

Ce guide explique, pour la soutenance, tout ce qu'il y a « sous le capot » : chaque notion est définie, sa prononciation est indiquée, son fonctionnement est expliqué simplement, puis son usage précis dans le projet (avec les valeurs réelles du code et du rapport).

> Convention de prononciation : entre crochets, une prononciation « à la française » facile à dire devant le jury, par exemple **CLAHE** [klè-ache] ou [cé-èl-a-ache-e].

---

0. Vue d'ensemble en une minute

1. La caméra fournit un flux d'**images** (frames).
2. Chaque image est envoyée à un **isolate** Dart (un fil d'exécution séparé) pour ne pas bloquer l'interface.
3. Dans l'isolate, **OpenCV** :
   - convertit l'image en matrice ;
   - renforce le contraste (**CLAHE**) ;
   - cherche les **contours** avec quatre méthodes (**Canny**, **seuillage adaptatif**, **saturation HSV + Otsu**, **gradient morphologique**) ;
   - garde le meilleur **quadrilatère** grâce à un **score géométrique** ;
   - le **redresse** par **homographie** en une image de 856 × 540 px.
4. Sur cette carte redressée, on mesure :
   - la **netteté** (variance du **Laplacien**) ;
   - la **luminosité** et les **reflets** ;
   - la **stabilité** des coins.
5. On vérifie la **bonne face** :
   - **recto** : visage (**cascade de Haar**), logo et drapeau (**template matching**) ;
   - **verso** : **code-barres**, **empreinte** (Laplacien), **ligne de séparation** (**transformée de Hough**).
6. Une **machine à états** décide de la capture : deux images bonnes consécutives → capture automatique.
7. Après capture : aperçu, recadrage manuel possible, **amélioration** (contraste, netteté), **page A4**, export galerie, historique.
8. Avatar-ID enchaîne ensuite : **test de vivacité**, **comparaison des photos**, **OCR**, enregistrement (hors de mon module).

---

1. Contexte et vocabulaire général

Terme	Prononciation	Signification
CIN	[cé-i-ène]	Carte d'Identité Nationale (tunisienne).
KYC	[ka-wa-i-cé] ou à l'anglaise	*Know Your Customer* :

Terme	Prononciation	Signification
	[kay-ouaï-si]	vérifier l'identité d'un client à distance (banques, services en ligne).
OCR	[o-cé-èr]	*Optical Character Recognition* : lecture automatique du texte d'une image. Fait par Avatar-ID après mon module.
Vivacité (*liveness*)	[laïv-nèss]	Vérifier qu'une vraie personne est présente en direct (et pas une photo/vidéo). Fait par Avatar-ID après mon module.
Autocapture	—	Déclenchement automatique de la photo, sans bouton, quand toutes les conditions sont réunies.
Recto / verso	—	Face avant (photo, logo, drapeau) / face arrière (code-barres, empreinte, ligne de séparation).
ID-1	[i-dé-un]	Format standard des cartes (norme ISO/IEC 7810) : 85,6 × 54 mm, rapport **ro = 1,586**.
SDK	[èss-dé-ka]	*Software Development Kit* : kit de développement prêt à l'emploi (ex. ML Kit).
ML Kit	[èm-èl kit]	Bibliothèque de Google ; son *Document Scanner* n'existe que sur Android → écarté.
Vision / VisionKit	—	Frameworks d'Apple pour iOS → écartés (il faudrait deux chaînes de détection).

2. Technologies

Flutter [flœ-teur] et Dart [dart]

- **Flutter** : framework de Google pour écrire **une seule application** qui tourne sur Android et iOS.
- **Dart** : le langage de Flutter.
- Le projet est entièrement en Flutter/Dart : un seul code pour les deux plateformes.

OpenCV [o-peune-cé-vé]

- *Open Source Computer Vision Library* : la bibliothèque de référence en vision par ordinateur (écrite en C++).
- Elle fournit les **primitives** : flou, Canny, contours, homographie, Haar, Hough, etc.

opencv_dart et FFI [èf-èf-i]

- **opencv_dart** est un *binding* : il permet d'appeler OpenCV (C++) depuis Dart.
- Le mécanisme utilisé s'appelle **FFI** (*Foreign Function Interface*) : Dart appelle directement du code natif compilé.
- Conséquence importante : les matrices OpenCV (**Mat**) vivent en **mémoire native**, que le ramasse-miettes de Dart ne gère pas. Il faut donc les libérer à la main (`dispose()`), sinon fuite mémoire à chaque image. Le projet libère systématiquement chaque Mat intermédiaire.

Mat [mat]

- Structure de données d'OpenCV pour une image : une **matrice** de pixels (lignes × colonnes × canaux).
- Une image couleur a 3 canaux (bleu, vert, rouge) ; une image en niveaux de gris, 1 canal.

Isolate Dart [aïsolète]

- Dart exécute l'interface sur un seul fil ; un calcul lourd sur ce fil fait **saccader l'aperçu**.
- Un **isolate** est un fil d'exécution séparé, avec **sa propre mémoire** ; on communique avec lui par **messages**.
- Dans le projet, `DetectionIsolateWorker` est un isolate **persistant** :
  - créé une fois ;
  - chargé une fois avec la cascade de Haar et les gabarits ;
  - interrogé image par image, chaque requête portant un identifiant de corrélation.
- **Pas de file d'attente** : tant qu'une image est en cours d'analyse (`_busy`), les nouvelles images caméra sont **ignorées**. Le débit s'adapte seul à la vitesse réelle, sans accumuler de retard.

Autres paquets

Paquet	Rôle
`camera`	Flux caméra et capture.
`permission_handler`	Demande de permission caméra.
`provider`	Gestion d'état (les *view-models* `ChangeNotifier`).
`path_provider`, `gal`	Stockage et export vers la galerie.
`image` (package:image)	Post-traitement en Dart pur (contraste, netteté, page A4).
`flutter_test`, `mocktail` [mok-tèil]	Tests ; *mocktail* crée des faux objets (*mocks*) pour simuler les plugins.

3. Architecture logicielle

Clean Architecture

- Principe de Robert C. Martin : séparer le code en **couches**, où les dépendances vont **vers le cœur métier**.
- Trois couches :
  - **presentation** : écrans et *view-models* (ce que voit l'utilisateur) ;
  - **domain** : entités et cas d'usage, **en Dart pur**, sans Flutter ni OpenCV ;
  - **data** : sources de données concrètes (détecteurs OpenCV, caméra, stockage), qui **implémentent** les interfaces du domaine.
- Intérêt : la logique du domaine (machine à états, décision de capture, géométrie) se **teste sans caméra ni émulateur**.

Feature-first [fi-tcheur feurst]

- Le code est rangé **par fonctionnalité**, chaque module ne prenant que les couches dont il a besoin :
  - `camera_capture`, `document_detection`, `image_quality`, `autocapture` ;
  - `image_postprocessing`, `result_preview`, `capture_history`.
- Deux modules transverses complètent l'ensemble : `core/services` et `shared/utils`.
- Avantages :
  - l'arborescence suit les six étapes du stage ;
  - modifier un module ne casse pas silencieusement un autre.

MVVM / view-model

- Un **view-model** expose l'état à l'écran et réagit aux événements, par exemple `DetectionViewModel` et `AutocaptureViewModel`.
- L'écran se redessine quand l'état change (`notifyListeners`).

---

4. Images, couleurs et formats

Notion	Prononciation	Explication
Pixel	—	Point de l'image, de valeur 0 (noir) à 255 (blanc) par canal.
Niveaux de gris	—	Un seul canal d'intensité ; la plupart des traitements se font en gris.
BGR	[bé-gé-èr]	Ordre des canaux couleur dans OpenCV (bleu, vert, rouge).
YUV420	[i-u-vé quatre-cent-vingt]	Format brut des caméras Android : **Y** = luminance, **U/V** = chrominance sous-échantillonnée. Le projet le

Notion	Prononciation	Explication
		réorganise en **NV21** puis le convertit en BGR.
BGRA	[bé-gé-èr-a]	Format brut des caméras iOS : BGR + canal alpha (transparence).
HSV	[ache-èss-vé]	*Hue, Saturation, Value* (teinte, saturation, valeur). La **saturation** distingue une carte colorée d'un fond gris ou blanc.
Histogramme	—	Combien de pixels ont chaque intensité (0–255). Sert au calcul de la **médiane** et à Otsu.
Médiane vs moyenne	—	La médiane est la valeur du milieu, la moyenne est la somme divisée par le nombre de pixels. Sur une scène très claire avec un peu de sombre, elles diffèrent beaucoup.
ROI	[èr-o-i] ou [roï]	*Region Of Interest* : zone de l'image où l'on cherche quelque chose (ex. coin haut-gauche pour le visage).

5. Pré-traitement

Flou gaussien [go-ssi-un]

- Remplace chaque pixel par une moyenne pondérée de ses voisins, avec un poids en forme de cloche (loi de Gauss).
- Il atténue le **bruit** avant de chercher des contours. Noyaux utilisés : 3 × 3 et 5 × 5.

Filtre bilatéral

- Flou qui **préserve les bords** : il ne mélange que les pixels voisins **et** de couleur proche.
- Il est utilisé dans la passe fine (paramètres 9, 75, 75).

CLAHE [klè-ache] — *Contrast Limited Adaptive Histogram Equalization*

- **Égalisation d'histogramme** : on étire les intensités pour utiliser toute la plage 0–255, ce qui augmente le contraste.
- **Adaptive** : l'opération se fait **par tuiles** (ici une grille 8 × 8), donc le contraste est renforcé **localement**, pas globalement.

- **Contrast Limited** : on **plafonne** l'amplification (*clip limit*) pour ne pas exagérer le bruit.
- Dans le projet :
  - clip limit **2,5** dans la passe fine, **3,5** dans la passe grossière ;
  - il fait ressortir le bord faible d'une **carte claire sur fond clair** ;
  - il est appliqué **aussi au gabarit** du logo et du drapeau, pour que les deux soient comparés dans la même représentation ;
  - il est appliqué avant le Laplacien de netteté.

---

6. Recherche du contour de la carte

Canny [ka-ni] (John Canny, 1986)

Détecteur de contours en quatre étapes : 1. **Flou** pour réduire le bruit. 2. **Gradient** (variation d'intensité) en x et en y, calculé avec des filtres de **Sobel** [so-bèl]. Un fort gradient signale un bord possible. 3. **Suppression des non-maxima** : on amincit les bords à 1 pixel d'épaisseur. 4. **Seuillage par hystérésis**, avec deux seuils : - au-dessus du seuil **haut** : bord certain ; - entre les deux : bord gardé seulement s'il touche un bord certain ; - sous le seuil **bas** : rejeté.

Dans le projet : - Les seuils sont **dérivés de la médiane** d'intensité : bas = 0,66 × médiane, haut = 1,33 × médiane. Une heuristique classique, qui s'adapte à la luminosité de la scène. - Garde-fous : le seuil bas est plafonné à 90 et le seuil haut a un plancher de 60. - **Le bug corrigé** : les seuils étaient calculés sur la **moyenne**. Sur une scène claire, la moyenne monte, les seuils deviennent trop hauts et le bord faible d'une carte claire disparaît.

Seuillage adaptatif (*adaptive threshold*)

- Binarise l'image (noir/blanc) avec un seuil **calculé localement**, à partir de la moyenne pondérée gaussienne d'un voisinage de 31 × 31.
- Utile quand l'éclairage n'est pas uniforme.

Méthode d'Otsu [ot-sou] (Nobuyuki Otsu, 1979)

- Choisit **automatiquement** le seuil qui sépare au mieux l'histogramme en deux classes : il minimise la variance à l'intérieur de chaque classe.
- Dans le projet, Otsu est utilisé :
  - sur la **saturation HSV** : passe « couleur » ;
  - sur le **gradient morphologique** : dernière passe.

Morphologie mathématique

On fait glisser un « élément structurant » (un petit rectangle de 3 × 3, 5 × 5, 9 × 9 ou 15 × 15 pixels) sur un masque binaire :

Opération	Effet
Dilatation	Épaissit les zones blanches (relie des morceaux de bord).

Opération	Effet
Érosion	Amincit les zones blanches (enlève les petits points).
Fermeture (*closing*)	Dilatation puis érosion : **bouche les trous** d'un contour sans trop l'épaissir.
Gradient morphologique	Dilatation moins érosion : fait ressortir les **bords**, en réagissant à de très petites différences locales.
Top-hat / black-hat	Font ressortir les fines structures claires ou sombres. Utilisés pour la **ligne de séparation** du verso.

Dans le projet, la dernière passe (gradient morphologique) sert de recours quand la carte et le fond ont presque le même niveau de gris, même après CLAHE.

Les « quatre passes » et la sortie anticipée

1. **Canny** (masque fermé par morphologie, avec une variante plus fine).
2. **Seuillage adaptatif**.
3. **Saturation HSV + Otsu**.
4. **Gradient morphologique + Otsu**.

On s'arrête dès qu'un candidat atteint un score de **0,65** : c'est la **sortie anticipée**, qui évite les passes coûteuses. Si aucun candidat n'atteint ce score, toutes les passes sont faites et on garde le meilleur.

Des contours au quadrilatère

1. **`findContours`** extrait les contours fermés du masque. Seuls les 10 plus grands sont gardés, et ils doivent couvrir **au moins 3,5 %** de l'image.
2. **`approxPolyDP`** simplifie chaque contour en polygone, avec une tolérance de 2 % du périmètre. C'est l'algorithme de **Douglas-Peucker** [douglass-peu-keur]. Si le polygone a **4 sommets** et est **convexe**, c'est un candidat net.
3. Sinon, **`minAreaRect`** donne le plus petit rectangle englobant (même incliné). Le score de ce candidat est **pénalisé : × 0,85**.
4. **Ordonnancement des coins** :
   - haut-gauche : plus petite somme x + y ;
   - bas-droit : plus grande somme ;
   - haut-droit : plus petite différence y − x ;
   - bas-gauche : plus grande différence.
5. **Aire du quadrilatère** : formule du lacet (*shoelace*), $\frac{1}{2} \times |\Sigma\ (x_i y_{i+1} - x_{i+1} y_i)|$.

Le score géométrique (Éq. 4.1 du rapport)

score = 0,8 × score_proportion + 0,2 × score_taille

- **score_proportion** = $\max(0, 1 - \min(|r - r_0|, |1/r - r_0|))$.

  - r est le rapport largeur/hauteur mesuré et r_0 = 1,586 celui du format ID-1.
  - Le terme en 1/r rend le score insensible à une carte vue tournée de 90°.
- **score_taille** compare l'aire du quadrilatère à une aire idéale de **30 %** de l'image : il monte linéairement jusqu'à 30 %, puis redescend au-delà.
- Un candidat n'est accepté qu'à partir de **0,30**.
- **Garde-fous** :
  - un quadrilatère collé au bord de l'image est rejeté ;
  - un quadrilatère qui couvre **plus de 75 %** de l'image est rejeté (sinon on pourrait « verrouiller » la table entière).
- **Resserrement de 1,5 %** : le quadrilatère final est légèrement rétréci vers son centre, pour compenser l'épaississement dû à la morphologie et éviter de voir un liseré de fond.

Recherche pleine image vs recherche suivie

- **Pleine image**, tant qu'aucune carte n'est suivie :
  - passe grossière sur l'image réduite à **45 %**, pour la vitesse ;
  - puis passe fine en pleine résolution, dans une marge de **15 %** autour de la carte trouvée.
- **Suivie**, une fois la carte trouvée : on ne cherche que dans une marge de **±20 %** autour du quadrilatère précédent. C'est beaucoup plus rapide et cela évite le tremblement d'une image à l'autre.

---

7. Redressement : homographie et warp

- Une **homographie** [o-mo-gra-fi] est une transformation de perspective, représentée par une matrice 3 × 3 (8 degrés de liberté).
- **Quatre paires de points** suffisent à la calculer : les 4 coins détectés et les 4 coins d'un rectangle cible.
- **`getPerspectiveTransform`** calcule cette matrice ; **`warpPerspective`** [ouorp] applique la transformation à l'image.
- Résultat : une carte **vue de face, à plat**, en **856 × 540 px**, une résolution qui respecte le rapport ID-1.
- Tous les contrôles suivants travaillent sur cette image redressée. C'est pour cela que le seuil de netteté **120** n'a de sens qu'à cette résolution.

---

8. Contrôles de qualité

Netteté : variance du Laplacien [la-pla-si-un]

- Le **Laplacien** est la **dérivée seconde** de l'image : il réagit fortement aux transitions brusques, c'est-à-dire aux bords nets.
- Sur une image **nette**, les réponses sont fortes et variées : **variance élevée**. Sur une image **floue**, les bords sont adoucis : **variance faible**.
- **Dans le projet** :

  - CLAHE d'abord, pour ne pas confondre une carte peu contrastée avec une carte floue ;
  - puis Laplacien, puis variance (écart-type au carré) ;
  - seuil : **variance ≥ 120**, à 856 × 540 px.

Luminosité et reflets

- **Moyenne** de l'intensité, dans **[60 ; 225]** : en dessous, l'image est trop sombre ; au-dessus, elle est trop lumineuse.
- **Fraction de pixels saturés**, c'est-à-dire ≥ 250 : au plus **12 %**. Au-delà, il y a un **reflet** sur le plastique, même si la moyenne paraît correcte.

Stabilité

- On mesure la **dérive moyenne des 4 coins** entre deux images, **divisée par la diagonale** du quadrilatère. Le seuil est ainsi indépendant de la résolution.
- Stable si la dérive reste ≤ **2,5 %** pendant **3** images consécutives.
- Réinitialisé après **2** images instables : une seule image bruitée ne casse pas la stabilité.

Lissage des signaux (`DetectionStabilizer`)

- Chaque signal (bord, visage, logo, drapeau, code-barres, empreinte, ligne) ne passe à « détecté » qu'après **2** images consécutives positives.
- Il ne retombe qu'après **3** images négatives.
- C'est un **anti-rebond** (*debounce*) : il évite le clignotement des indicateurs à l'écran.

---

9. Machine à états de l'autocapture

État	Signification
searching	Aucune carte détectée.
poorQuality	Carte détectée, mais au moins un contrôle échoue (flou, sombre, reflet, instable, mauvaise face). Un message guide l'utilisateur.
holding	Tous les contrôles sont bons ; on **maintient**.
triggerCapture	Le maintien est atteint (**2** images bonnes consécutives) : la capture est déclenchée.
captured	Capture faite ; état final jusqu'au passage à la face suivante.

- Retours :
  - carte perdue → **searching** ;
  - contrôle raté → **poorQuality** ;
  - échec de la capture → réinitialisation.
- Le maintien est un **compteur d'images, pas une minuterie** : on évite un déclenchement sur un seul instant favorable.

- La décision est isolée dans un **cas d'usage du domaine** (`EvaluateCaptureReadiness`), en Dart pur, donc testable sans OpenCV.

---

10. Reconnaissance du recto et du verso

Cascade de Haar [ar] (Viola & Jones [vi-o-la et djonz], 2001)

- **Caractéristiques de Haar** : différences de somme d'intensité entre rectangles adjacents clairs et sombres. Par exemple, les yeux sont plus sombres que les joues.
- **Image intégrale** : une astuce qui permet de calculer la somme de n'importe quel rectangle en **4 opérations**, ce qui rend le calcul très rapide.
- **AdaBoost** [ada-boust] : sélectionne les meilleures caractéristiques et les combine en classifieurs.
- **Cascade** : une suite de classifieurs du plus simple au plus exigeant. Une zone rejetée tôt n'est plus examinée, d'où la rapidité.
- **`detectMultiScale`** balaie l'image à plusieurs échelles :
  - **scaleFactor = 1,05** : on agrandit la fenêtre de 5 % à chaque étape ;
  - **minNeighbors = 8** : il faut 8 détections qui se recoupent pour valider un visage, ce qui limite les faux positifs.
- **Dans le projet** :
  - on utilise le fichier standard `haarcascade_frontalface_default.xml` d'OpenCV, **pré-entraîné**, sans aucun entraînement sur la CIN ;
  - la recherche se fait dans la zone **haut-gauche** (35 % de la largeur, marge haute de 15 %), sur **4 rotations** (0°, 90°, 180°, 270°) ;
  - la rotation qui trouve le visage donne **l'orientation** de la carte.

Template matching (corrélation de gabarit)

- On fait glisser une **image de référence** (le gabarit : logo, drapeau) sur une région, et on calcule à chaque position un score de ressemblance.
- **TM_CCOEFF_NORMED** : coefficient de corrélation normalisé, entre −1 et 1. La normalisation le rend insensible à un changement **global** de luminosité ou de contraste.
- **Dans le projet** :
  - seuil de **0,6** ;
  - recherche dans des régions fixes de la carte orientée ;
  - CLAHE appliqué au gabarit **et** à la région, pour corriger un éclairage non uniforme.

Détecteur de code-barres

- Le `BarcodeDetector` d'OpenCV **localise** le code-barres, sans décodage (inutile ici).
- On vérifie aussi **où** il se trouve : un code-barres est trouvé quelle que soit la rotation, donc sa **position** est l'indice d'orientation le plus fiable.

Empreinte digitale

- La **variance du Laplacien** est calculée dans la zone de l'empreinte (bas-droite) ; elle doit être dans **[150 ; 4 000]**.

- Trop faible : la zone est vide. Trop forte : bruit ou reflet.

Transformée de Hough [haf] probabiliste (Matas et al., 2000)

- Elle sert à détecter des **droites** dans une carte de contours. Chaque point de bord « vote » pour toutes les droites qui pourraient passer par lui ; les droites qui reçoivent beaucoup de votes existent vraiment.
- La version **probabiliste** (`HoughLinesP`) travaille sur un échantillon de points et renvoie directement des **segments**.
- **Dans le projet** :
  - la recherche est limitée au **bas de la carte** (35 % de la hauteur), sinon les lignes de texte ou les bords du code-barres concurrencent la vraie ligne ;
  - le segment doit faire au moins **45 %** de la largeur et être à moins de **10°** de l'horizontale ;
  - la carte de contours combine Canny, le gradient de saturation (Sobel) et le top-hat/black-hat.

Vote sur 4 rotations (verso)

- Pour chaque rotation, on calcule un score :
  - +2 si le code-barres est trouvé **en bonne position** ;
  - +1 si l'empreinte est plausible ;
  - +1 si la ligne est trouvée.
- La rotation la mieux notée gagne.
- Une correction finale de **+180°** compense une convention de position vérifiée sur une vraie carte.

Anti-doublon

- Après la capture du recto, l'application attend le verso.
- Seule une face qui réunit **les trois indices** du verso peut déclencher la capture : représenter le recto ne déclenche rien.

---

11. Après la capture

Amélioration d'image (`CardImageEnhancer`)

- **Contraste** légèrement renforcé.
- **Netteté** par convolution avec un noyau 3 × 3 de type Laplacien :

$$\begin{pmatrix} 0 & -a & 0 \\ -a & 1+4a & -a \\ 0 & -a & 0 \end{pmatrix}$$

  avec **a = 0,6**. Chaque pixel est accentué par rapport à ses 4 voisins.
- **Mode scan** : niveaux de gris, appliqué en dernier pour garder la couleur pendant les autres traitements.

- Réalisé avec **package:image**, en Dart pur, sans OpenCV.

Page A4

- Recto et verso empilés sur une page blanche **A4 portrait à 300 dpi**, soit **2480 × 3508 px**, comme un scanner.
- 300 dpi signifie 300 points par pouce ; une feuille A4 fait 8,27 × 11,69 pouces.

Export et historique

- Les images sont exportées dans la galerie (paquet `gal`), avec des noms horodatés.
- Un historique local permet de consulter, partager ou supprimer les captures.
- Le module transmet à Avatar-ID le **recto et le verso** (en `Uint8List`, des octets d'image) et la page A4.

---

12. Tests

Palier	Contenu	Où ça tourne
A	Logique pure Dart : machine à états, décision de capture, stabilité	Partout (`flutter test`)
B	Plateforme simulée par **mocks** (caméra, permissions, stockage)	Partout, sans appareil
C	OpenCV réel sur **images synthétiques** dessinées dans le test (rectangles, dégradés)	Machine avec le binaire natif OpenCV
D	Cartes réelles, sur téléphone (visage, code-barres, empreinte, ligne, parcours complet)	Protocole manuel

- Les 28 fichiers de test se répartissent en 15 de palier A, 6 de B et 7 de C. La dernière campagne a exécuté **177 cas, tous réussis**.
- Un **mock** est un faux objet qui imite un plugin, pour tester sans le vrai matériel.
- Une **image synthétique** est une image générée par le test (un rectangle blanc sur fond noir, par exemple) dont on connaît la bonne réponse.

---

13. Les chiffres à connaître par cœur

Valeur	Signification
6 semaines (30 jours ouvrés)	Durée du stage.
< 100 ms / **92–100 ms**	Cible / mesure du temps de traitement d'une image dans l'isolate.
≥ 90 % / **92 %**	Cible / mesure du recto/verso bien reconnus.
≈ 2 s	Cible et mesure : de la carte stabilisée à la

Valeur	Signification
	capture.
177	Cas de test automatisés réussis.
4 passes	Canny, adaptatif, saturation + Otsu, gradient morphologique.
0,66 / 1,33	Facteurs des seuils de Canny par rapport à la médiane.
0,30 / 0,65	Score minimal accepté / sortie anticipée.
0,8 / 0,2	Poids proportion / taille du score.
1,586	Rapport ID-1 (85,6 / 54).
±20 %	Marge de recherche suivie.
75 %	Aire maximale acceptée.
856 × 540 px	Résolution de la carte redressée.
120	Seuil de netteté (variance du Laplacien).
[60 ; 225] et **12 %**	Luminosité moyenne / pixels saturés tolérés.
2,5 %, **3** images	Dérive maximale des coins / images stables requises.
2 images	Maintien avant capture.
1,05 / 8	Haar : facteur d'échelle / voisins minimums.
0,6	Seuil du template matching.
[150 ; 4 000]	Bande de variance de l'empreinte.
2480 × 3508 px	Page A4 à 300 dpi.

14. Références

- Canny 1986 (contours) ; Otsu 1979 (seuillage) ; Zuiderveld 1994 (CLAHE) ; Viola & Jones 2001 (cascade de Haar) ; Matas et al. 2000 (Hough probabiliste).
- Hartley & Zisserman 2004 (homographie) ; Martin 2017 (Clean Architecture) ; ISO/IEC 7810:2019 (format ID-1).
- Voir la bibliographie de `Rapport_CIN.pdf`.